

PROGETTO IUM-TWEB

“Award Explorer”

Anno 2024/2025

Luis Andrè Barrios Luna Victoria (1054511)

Israa Moumane (1054651)

Irene Rosa(1083317)



UNIVERSITÀ
DI TORINO

Introduzione

Report realizzato da Israa, Irene e Luis per descrivere lo sviluppo del progetto di IUM-TWEB da 12CFU nell'anno accademico 2024/2025:

TWEB(6CFU) Consiste nello sviluppo di un'applicazione web che utilizza dataset in 2 server (NoSQL in MongoDB e SQL in PostgreSQL) collegati attraverso il Main Server per mostrare dati su film e i rispettivi attori.

IUM(3CFU) consiste in un'analisi di dati su larga scala usando Jupyter Notebooks (python) su film e relativi attori, paesi di rilascio e studi di produzione. Nel report a seguito si mostra come sono stati manipolati i dati con l'uso di librerie specifiche.

Technical tasks TWEB

database, servers e implementazione delle query

soluzione implementata

La web-app è stata realizzata implementando 2 server che contengono i dati e 1 mainserver che li collega, così organizzati:

JavaServer:

Contiene i dati del main dataset collegato con PgAdmin (movies.csv, ...), le API sono gestite con Java Spring boot(JPA). La table "Movies" ha come chiave primaria "id" che funge da Foreign key nelle table countries, actors, crew, posters, releases, genres, languages e themes.

Ogni table ha:

- una classe sua *tableName.java* (es. Actors.java) dove se ne definiscono i fields, i get and set e la chiave primaria composta nella classe *idtableName.java*;
- una classe *tableNameRepository.java* (es. ActorsRepository.java) in cui si definiscono le query fatte sulla tabella;
- una classe *tableNameService.java* (es. ActorsService.java) che chiama le query in repository;
- una classe *tableNameController.java* (es ActorsController.java) che riceve le richieste HTTP e usa *tableNameService.java* per chiamare le query.

La table "movies" ha una classe extra chiamata *moviesDTO.java* che serve a trasferire gli oggetti di movies.

OtherServer:

Contiene i dati del additional dataset collegato con MongoDB (the_oscars_awards.csv, rotten_tomatoes_reviews.csv che sono caricati su Mongo in questo modo: il database si chiama Award_Explor e ha le sottocartelle

rottenTomatoesReviews e theOscarAwards → screenshots in cartella report) usando mongoose nel file Database/database.js.

Le table sono state definite in Model/*tableName*.js e le query sono in Controller/controllersData.js.

MainServer:

Gestisce le richieste dell'utente collegando e facendo da intermediario tra il front-end e i server, "smista" le richieste dell'utente inoltrandole al server appropriato, prende i dati dal server e li mostra all'utente .

Per gestire i dati di grandi dimensioni con più efficienza abbiamo usato axios e le "hash-table" per evitare "colli di bottiglia" nel caricamento.

Con l'implementazione della chatbox con socket.io si ha la possibilità di chattare con gli altri utenti di argomenti inerenti ai film restando in anonimo o inserendo il nickname nella pagina di accesso in alto a destra della navbar.

Per evitare molteplici chiamate axios che possono causare rallentamenti abbiamo deciso di fare tutte le query attraverso movies perché è la "superclasse" che collega tutte le tabelle nel JavaServer.

Difficoltà affrontate

Prelevare i dati ottenuti dalle query e mostrarli correttamente nel front-end usando le handlebars e caricare le table in pgAdmin.

Requisiti

database in postgres → JavaServer

database in mongo → OtherServer

main server in express → MainServer

uso di Java springboot JPA → JavaServer

uso di axios → chiamate in MainServer

socket.io → chatbox in MainServer

Limitazioni

La query che recupera i film più apprezzati ha un limite di 9 film e in caso di necessità di vederne di più il caricamento della pagina potrebbe impiegare più tempo a causa dei poster.

Front-end

Il front-end è stato realizzato usando html, css, bootstrap, handlebars e javascript. Con bootstrap abbiamo realizzato un design standard che ha necessitato di alcune personalizzazioni in css, abbiamo usato le handlebars per rendere la web app più dinamica in modo che non necessiti il caricamento di nuove pagine molto simili. Nel layout.hbs abbiamo messo come standard la navbar e il footer mentre la searchbar appare solo nelle pagine che in javascript hanno showsearch = true perchè la pagina di accesso non ne ha bisogno e la pagina oscar ha una searchbar diversa (causa della diversità della query).

Requisiti

uso di html, css → personalizzazioni e scheletro delle pagine

bootstrap → componenti standard come la navbar, footer e le cards

handlebars → tutte le pagine sono .hbs e usano i componenti nella directory partials (searchbar, footer, navbar, horizontalCard ...)

javascript → come cambiano le pagine dopo i vari click e mantiene l'eventuale nome inserito dall'utente nella pagina accedi permettendo il log-out

difficoltà riscontrate

chiamare correttamente i componenti handlebars e inserire i dati dopo l'uso dei search

limitazioni

il design è chiaro e non presenta la possibilità di uso della dark-mode per facilitarne l'utilizzo di notte.

Technical tasks IUM

Soluzione implementata

Le analisi sono suddivise in 3 file per argomento chiave: uno per countries (countries.ipynb), uno per actors (actors.ipynb) e uno per movies (movies.ipynb). Abbiamo analizzato i dati dei databases caricati su pgAdmin, già utilizzati per Tweb. Abbiamo collegato questi database ai Jupyter Notebook. Le analisi vengono mostrate tramite dei grafici di vario tipo (mappe, istogrammi ecc.), sia interattivi che non. I vari Jupyter Notebook seguono il seguente schema:

- importazione delle librerie;
- collegamento con pgAdmin e prelievo delle tabelle di interesse;
- visualizzazione di tabelle filtrate per determinate analisi;
- grafico relativo alle tabelle filtrate precedentemente;
- commento del grafico.

Gli ultimi 3 punti vengono ripetuti per ciascun grafico. Ogni blocco e quasi ogni riga di codice è stata adeguatamente commentata.

requisiti

Uso di vari tipi di grafici e Jupyter notebooks → abbiamo usato sia grafici statici che grafici interattivi

difficoltà riscontrate

Capire quali fossero i grafici più adatti a certe rappresentazioni, capire come combinare i dati per fare le analisi e qualche pc ha faticato a fare il render di alcuni grafici.

limitazioni

Alcuni grafici risultavano illeggibili per la vasta quantità di voci avendo utilizzato tabelle immense.

Conclusioni

TWEB: Lavorare su questo progetto ci ha insegnato cosa fa il full-stack developer, come avvenga la gestione delle API provenienti da server di natura diversa e mostrato l'utilità dell'AI, senza la quale avremmo faticato/impiegato molto tempo a capire il significato degli errori sui server (es. regole CORS).

IUM: Ha mostrato come la manipolazione dei dati possa raccontare storie diverse e come mostri trends che andrebbero inosservati.

Suddivisione del lavoro

Israa Moumane:

TWEB: lavorato in collaborazione su molte parti del progetto con i teammates + Front-end oscar, log in, chat-box, fix IU generali e query axios sul mainServer, create API su repository/controllers/service con query/id delle tabelle languages, movies e posters nel java server, query per oscar sul otherServer.

IUM: Jupyter Notebook di actors.

Luis André Barrios Luna Victoria:

TWEB: Responsabile della creazione di due database con meccanismi di backup, gestendo i dati attraverso *JPA* per le entità *actors*, *crews*, *genres* e *countries*. Sul *Main Server*, si occupa della gestione di pagine e componenti *Handlebars (HBS)* e *JavaScript*, integrando le API tramite *Axios*. Inoltre, è relatore della documentazione tecnica del progetto, curando la *JavaDoc* e la *Swagger Documentation* per garantire una chiara descrizione delle funzionalità e dell'architettura del sistema. Lavora a stretto contatto con il team, interagendo frequentemente sui componenti per assicurare un'integrazione efficace e un flusso di lavoro ottimizzato.

IUM: Jupyter Notebook di movies.

Irene Rosa:

TWEB: lavorato spesso in contatto con gli altri componenti del gruppo per varie parti del progetto, aiutato a caricare le tabelle su pgAdmin, gestito i dati attraverso JPA per l'entità releases, studios e themes, nel main server gestito le pagine, dei componenti Handlebars, Javascript, le chat con i socket.io.

IUM: Jupyter Notebook di countries.

Informazioni Extra

Tweb: Cartelle cors configuration necessarie a causa di violazione del regolamento cors nella comunicazione tra localhost:3000, localhost:3001 e localhost:8080, e necessario cambiare le passwords nel file applicationproperties in Java Server.

Ium: github non mostra i grafici interattivi in anteprima ma in Pycharm si.

Bibliografia

TWEB

- uso di codice presentato a lezione;
- uso di AI per capire il significato degli errori inerenti ai server e adattamento di frammenti di codice.

IUM

- uso di codice presentato a lezione;
- uso di AI per realizzazione/miglioramento dei grafici.